PROGRAMMING & CS FUNDAMENTALS

SOLID Principles Visual Guide

Five design rules —
each with a before/after that shows the payoff.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

S — Single Responsibility

One class, one reason to change.

```
# BEFORE
class Report:
    def generate(self): ...
    def save_to_db(self): ...
    def email(self): ...
    def print_pdf(self): ...
```

```
# AFTER
class Report:        # data only
    def generate(self): ...
class ReportSaver: ...   # persistence
class ReportMailer: ...  # delivery
```

- 'Reason to change' = an axis of requirements: formatting vs storage vs delivery.
- Smaller classes are easier to test, name, and delete.
- Smell: a class whose methods never touch the same fields — it's two classes already.

STEP 2/6

O — Open / Closed

Open for extension, closed for modification.

```
# BEFORE
def price(o):
    if o.type == 'book': ...
    elif o.type == 'food': ...
    elif o.type == 'NEW': ...  # edit me!
```

```
# AFTER
class Pricer:          # interface
    def price(self, o): ...
class BookPricer(Pricer): ...
# new type = new class, zero edits
```

- New behavior arrives as NEW code, not edits to old code — old paths stay proven.
- Polymorphism is the mechanism: the if-chain becomes a class hierarchy.
- Plugin architectures are OCP at scale: core closed, extensions open.

STEP 3/6

L — Liskov Substitution

Subclasses must honor the base contract — no surprises.

```
# BEFORE
class Rect:
    def set_w(self, w): ...
    def set_h(self, h): ...
class Square(Rect):
    def set_w(self, w):  # sets BOTH!
        ...              # breaks callers
```

```
# AFTER
class Shape:            # shared interface
    def area(self): ...
class Rect(Shape): ...
class Square(Shape): ...  # siblings, not parent/chi
```

- The classic violation: Square is-a Rectangle mathematically, but NOT behaviorally.
- Test: any code using the base must work unchanged with the subclass.
- Violations hide in overridden methods that weaken preconditions or strengthen postconditions.

STEP 4/6

I — Interface Segregation

No client should depend on methods it doesn't use.

```
# BEFORE
class Worker:  # fat interface
    def work(self): ...
    def eat(self): ...
class Robot(Worker):
    def eat(self): raise ...  # forced li
```

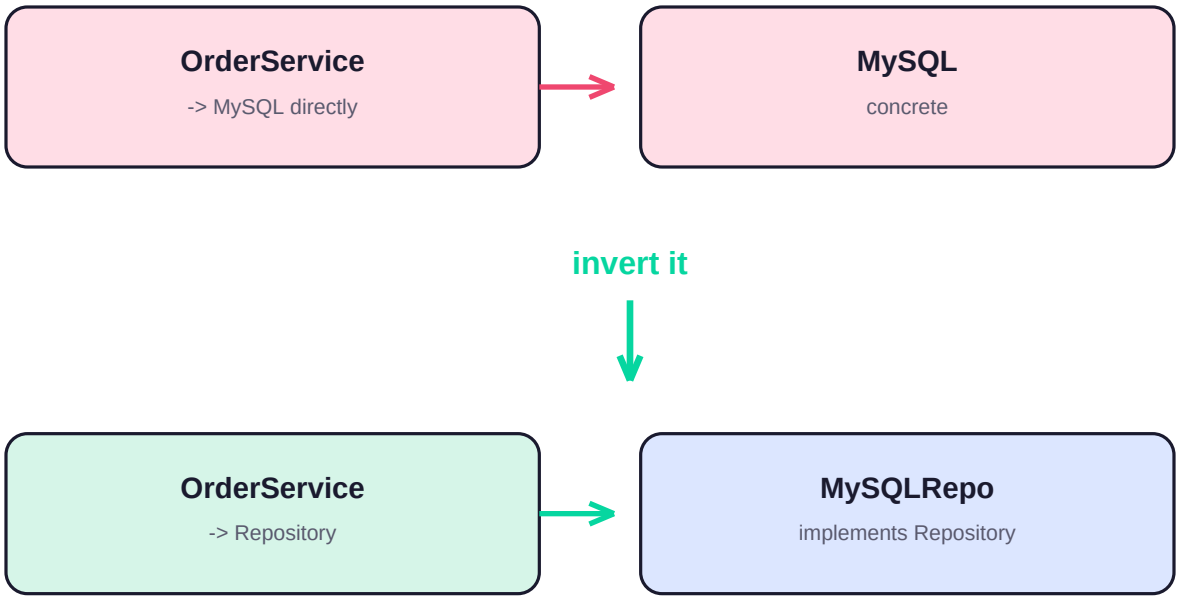
```
# AFTER
class Workable:          # small interfaces
    def work(self): ...
class Eatable:
    def eat(self): ...
class Robot(Workable): ...  # only what it needs
```

- Fat interfaces force empty/throwing methods — lies in the type system.
- Small role interfaces compose freely: Workable + Eatable as needed.
- Same idea as Unix tools: do one thing, combine via composition.

STEP 5/6

D — Dependency Inversion

Depend on abstractions, not concretions.




```
# constructor injection: pass the abstraction in
svc = OrderService(repo=MySQLRepo())   # or FakeRepo() in tests!
```

- High-level policy (orders) must not depend on low-level detail (MySQL).
- Testing payoff: swap the real repo for a fake — no database in unit tests.
- The interface is owned by the HIGH-level module — it declares what it needs.

STEP 6/6

SOLID Together

Five rules, one direction: change gets cheaper.

- SRP splits by reason to change; OCP makes change additive; LSP keeps substitution safe.
- ISP keeps contracts small; DIP points dependencies at abstractions.
- SOLID serves MAINTAINABILITY — for a 100-line script, it's over-engineering. Apply by scale.
- Smells that call for SOLID: shotgun surgery, rigid if-chains, untestable constructors.
- Principles guide, not laws: a pragmatic violation with a comment beats dogmatic purity.
- Pair with: composition over inheritance, dependency injection, and small functions.

STEP ✓I✓

Key Takeaways

SOLID Principles — Remember

- S: one class, one reason to change — split by axes of requirements.
- O: extend with new code, don't edit proven code — polymorphism over if-chains.
- L: subclasses must honor the base contract (Square is not a behavioral Rectangle).
- I: small role interfaces beat fat ones — no forced empty methods.
- D: depend on abstractions; inject concretions — testability is the payoff.

Keep exploring

Pair this guide with the matching video walkthroughs
and hands-on labs at interstitiumlabs.dev